Sales Project

E-commerce

Description

This project focuses on analyzing an e-commerce sales dataset to derive meaningful business insights related to customer behavior, product performance, and revenue trends. The dataset consists of multiple relational tables including customers, orders, products, order items, and payments.

The objective of this project is to perform end-to-end data analysis using SQL by writing queries ranging from basic to advanced levels.

The analysis helps in understanding key business metrics such as total revenue, customer spending patterns, sales trends, and product performance.

Key objectives

- Analyze customer purchasing behavior
- Identify top-performing customers and products
- Track sales trends over time (daily, monthly)
- Evaluate payment success and failure rates
- Perform Pareto analysis (top 20% customers contributing to 80% revenue)

Tools & Technologies

- SQL (Joins, Aggregations, Window Functions, CTEs)
- Database: SQL Server
- (Optional) Power BI for dashboard visualization

Business Insights

- A small percentage of customers contribute to a large portion of total revenue
- Certain product categories generate higher sales consistently
- Sales trends help identify peak business periods
- Payment success rate directly impacts revenue realization

Outcome

This project demonstrates strong SQL skills including data extraction, transformation, and analysis. It also showcases the ability to convert raw data into actionable business insights, making it suitable for roles like Data Analyst or SQL Developer.

Mobile-first design implementation

The project uses the following tables:

- Customers → customer details (name, city, etc.)
- Orders → order-level data (date, payment mode)
- Products → product information (category, price)
- Order_Items → transaction-level data (quantity, amount)
- Payments → payment status (success, failed, pending)

1.Find the total number of customers.

```
select COUNT(*) as total_number
from customers;
```

total_number
34

2. Find the total number of orders.

```
select COUNT(*) as total_num_of_orders
from orders;
```

total_num_of_orders
34

Insights

“Comparing total customers with total orders helps evaluate customer engagement and repeat purchase behavior.”

- 3. Calculate the total sales amount.
- 4. Count the number of unique cities.
- 5. Find the minimum and maximum product price.

1

```sql
select
    SUM(o.total_amount) as total_sales
from
    order_items o
join payments p on p.order_id = o.order_id
where
    p.payment_status = 'Success';
```

total_sales
301400.00

2

```sql
select
    COUNT(distinct city) as total_unique_city
from customers;
```

total_unique_city
5

3

```sql
select
    MIN(price) as minimum_price,
    MAX(price) as maximum_price
from products;
```

minimum_price	maximum_price
500.00	70000.00

Insights

"Total sales reflect overall business revenue and indicate the company's financial performance."

"The number of unique cities shows market reach and geographical expansion of the business."

"The price range helps understand product positioning and customer affordability segments."

- 6. Find the total number of orders for each customer.
- 7. Calculate total spending for each customer.
- 8. Find city-wise total sales.

Insights

"Analyzing customer orders, spending, and city-wise sales helps identify loyal high-value customers and top-performing regions for business growth."

6

```sql
select
    customer_id,
    COUNT(order_id) as total_orders
from orders
group by customer_id;
```

customer_id	total_orders
1	1
2	1
3	1
4	1
5	1
6	1
7	1

7

```sql
select
    o.customer_id,
    SUM(oi.total_amount) as total_spending
from order_items oi
join orders o on oi.order_id = o.order_id
group by
    o.customer_id;
```

customer_id	total_spending
3	10000.00
4	3500.00
5	60000.00
6	15000.00
7	6000.00
8	8000.00
9	4000.00
10	4000.00
11	70000.00
12	50000.00
13	10000.00
14	3500.00
15	60000.00
16	15000.00
17	4000.00
18	8000.00
19	4000.00
20	2400.00
21	70000.00

8

```sql
select
    c.city,
    SUM(oi.total_amount) as total_sales_per_city
from orders o
join order_items oi
    on o.order_id = oi.order_id
join customers c
    on o.customer_id = c.customer_id
join payments p
    on o.order_id = p.order_id
where p.payment_status = 'Success'
group by c.city;
```

city	total_sales_per_city
Bangalore	15000.00
Delhi	170000.00
Hyderabad	6400.00
Mumbai	110000.00

9. Calculate category-wise total revenue.

10. Find the top 5 highest spending customers.

9

```sql
select
        p.category,
        SUM(oi.total_amount) as total_revenue
from order_items oi
join products p
    on oi.product_id = p.product_id
join payments ps
    on oi.order_id = ps.order_id
where payment_status = 'Success'
group by p.category;
```

10

```sql
select top 5
            c.customer_id,
            c.customer_name,
            SUM(oi.total_amount) as high_spending_customer
from order_items oi
join orders o
    on oi.order_id = o.order_id
join customers c
    on o.customer_id = c.customer_id
join payments p
    on oi.order_id = p.order_id
where payment_status = 'Success'
    group by c.customer_name,
            c.customer_id
    order by high_spending_customer
desc;
```

9

category	total_revenue
Electronics	248000.00
Fashion	13400.00
Home	30000.00
Sports	10000.00

10

customer_id	customer_name	high_spending_customer
1	Customer_1	70000.00
11	Customer_11	70000.00
2	Customer_2	50000.00
12	Customer_12	50000.00
6	Customer_6	15000.00

Insights

"Identifying top spending customers and category-wise revenue helps focus on high-value customers and the most profitable product segments."

11. Calculate the average order value per customer.

insights

“Average order value highlights customer spending patterns and helps optimize pricing, bundling, and upselling strategies.”

```sql
select
        customer_id,
        AVG(order_value) as average_ordervalue
FROM(
    select
            o.customer_id,
            o.order_id,
            SUM(oi.total_amount) as order_value
    from orders o
    join order_items oi
        on o.order_id = oi.order_id
        where o.order_id IN(
            select order_id
            from payments
            where LOWER(payment_status) = 'success'
        )
        group by o.customer_id,
                o.order_id
) t
group by customer_id;
```

customer_id	average_ordervalue
1	70000.000000
2	50000.000000
4	3500.000000
6	15000.000000
7	6000.000000
9	4000.000000
10	4000.000000
11	70000.000000
12	50000.000000
14	3500.000000
16	15000.000000
17	4000.000000
19	4000.000000
20	2400.000000

12. Find product-wise total quantity sold.

```sql
select
        p.product_name,
        SUM(quantity) as product_wise_quantity_sold
from order_items oi
join orders o
    on oi.order_id = o.order_id
join products p
    on oi.product_id = p.product_id
join payments ps
    on oi.order_id = ps.order_id
where o.order_id IN(
    select
        order_id
    from payments
    where LOWER(payment_status) = 'success'
) group by p.product_name;
```

product_name	product_wise_quantity_sold
Adidas Jacket	2
Cricket Bat	5
Dining Table	2
Headphones	4
iPhone 14	2
Samsung TV	2
T-Shirt	8

Insights

"Analyzing product-wise quantity sold helps optimize inventory, focus on high-demand products, and improve sales strategy."

13. Generate a month-wise sales report

```sql
select
        YEAR(o.order_date) as yearcount,
        MONTH(o.order_date) as month_count,
        DATENAME(MONTH,o.order_date) as monthname,
        SUM(oi.total_amount) as total_sales_per_month
from orders o
    JOIN order_items oi
        on o.order_id = oi.order_id
    join payments p
        on oi.order_id = p.order_id
    where o.order_id IN(
        select
            order_id
        from payments
        where
            LOWER(payment_status) = 'success'
    )
group by
        YEAR(o.order_date),
        month(o.order_date),
        DATENAME(month,o.order_date)
order by yearcount;
```

yearcount	month_count	monthname	total_sales_per_month
2024	1	January	301400.00

Insights

"Month-wise sales trends help identify seasonal patterns, peak periods, and support better forecasting and planning."

14. Count total orders based on payment mode.

```sql
select
        payment_mode,
        COUNT(*) as total_orders_payment_based
from orders
group by payment_mode;
```

payment_mode	total_orders_payment_based
Card	10
COD	7
NetBanking	3
UPI	14

Insights

"Analyzing orders by payment mode helps optimize payment options, improve user experience, and increase transaction success rates."

15. Compare failed vs successful payments.

```sql
SELECT
        payment_status,
        COUNT(*) as comparing
from payments
group by payment_status;
```

payment_status	comparing
Failed	2
Pending	4
Success	14

Insights

"Payment success vs failure analysis helps identify revenue leakage and improve payment system efficiency."

16.Find the top 10 customers based on total spending.

```sql
select top 10
      c.customer_id,
      c.customer_name,
      SUM(total_amount) as total_spending
from order_items oi
    JOIN orders o
      on oi.order_id = o.order_id
    join customers c
      on o.customer_id = c.customer_id
    JOIN payments p
      on oi.order_id = p.order_id
    WHERE o.order_id IN(
        select
              order_id
        from payments
        where LOWER(payment_status) = 'success'
    )
group by c.customer_id,c.customer_name
order by total_spending
desc;
```

customer_id	customer_name	total_spending
1	Customer_1	70000.00
11	Customer_11	70000.00
12	Customer_12	50000.00
2	Customer_2	50000.00
16	Customer_16	15000.00
6	Customer_6	15000.00
7	Customer_7	6000.00
10	Customer_10	4000.00
9	Customer_9	4000.00
17	Customer_17	4000.00

Insights

“Identifying top spending customers helps target high-value users for retention, loyalty programs, and personalized marketing.”

17. Identify repeat customers (customers with multiple orders).

```sql
SELECT
        c.customer_name,
        o.customer_id,
        COUNT(o.order_id) as repeatcustomers
from orders o
    join customers c
        on o.customer_id = c.customer_id
group by
        o.customer_id,
        c.customer_name
HAVING COUNT(o.order_id) > 1;
```

zero repeated customer

Insights

“Identifying repeat customers helps measure retention rate and focus on strategies that drive long-term customer value.”

18. Find customers whose orders do not have successful payments.

```sql
select
        c.customer_name,
        o.order_id
from orders o
    join customers c
        on o.customer_id = c.customer_id
    join payments p
        on o.order_id = p.order_id
WHERE o.order_id NOT IN(
    select
        order_id
    from payments
    where lower(payment_status) = 'success'
)
group by o.order_id,c.customer_name
order by o.order_id;
```

customer_name	order_id
Customer_3	1003
Customer_5	1005
Customer_8	1008
Customer_13	1013
Customer_15	1015
Customer_18	1018

Insights

"Identifying customers with failed or pending payments helps detect revenue loss and improve payment recovery strategies."

19.Identify products that were never sold.

```sql
select
    p.product_id,
    p.product_name
from products p
LEFT JOIN order_items oi
        on p.product_id = oi.product_id
where oi.product_id is NULL;
```

product_id	product_name
111	Refrigerator
112	Washing Machine
113	Sofa Set
114	Bed
115	Football
116	Badminton Racket
117	Smart Watch
118	Bluetooth Speaker
119	Jeans
120	Shirt
121	Tablet
122	Gaming Console
123	Office Chair
124	Study Table
125	Tennis Ball
126	Yoga Mat
127	Earbuds
128	Camera
129	Jacket
130	Sneakers
131	Mixer Grinder
132	Air Conditioner
133	Bookshelf

Insights

"Identifying unsold products helps optimize inventory, improve product positioning, and reduce holding costs."

20. Analyze daily sales trends.

```sql
SELECT
      order_date,
      dailysales,
      LAG(dailysales) OVER(order by order_date) as previous_sale,
      dailysales - lag(dailysales) OVER(order by order_date) as DIFFERENCE,

      CASE
          when dailysales > LAG(dailysales) OVER(order by order_date)
          THEN 'increase'
          when dailysales < LAG(dailysales) over (order by order_date)
          THEN 'decrease'
          else 'nochange'
      end as trends
FROM (
    select
          o.order_date,
          SUM(total_amount) as dailysales
    from order_items oi
       join orders o
          on oi.order_id = o.order_id
       join payments p
          on oi.order_id = p.order_id
       where o.order_id IN(
           select order_id
           from payments
           where LOWER(payment_status) = 'success'
       )
       group by o.order_date
) t;
```

order_date	dailysales	previous_sale	DIFFERENCE	trends
2024-01-01	70000.00	*NULL*	*NULL*	nochange
2024-01-02	50000.00	70000.00	-20000.00	decrease
2024-01-04	3500.00	50000.00	-46500.00	decrease
2024-01-06	15000.00	3500.00	11500.00	increase
2024-01-07	6000.00	15000.00	-9000.00	decrease
2024-01-09	4000.00	6000.00	-2000.00	decrease
2024-01-10	4000.00	4000.00	0.00	nochange
2024-01-11	70000.00	4000.00	66000.00	increase
2024-01-12	50000.00	70000.00	-20000.00	decrease
2024-01-14	3500.00	50000.00	-46500.00	decrease
2024-01-16	15000.00	3500.00	11500.00	increase
2024-01-17	4000.00	15000.00	-11000.00	decrease
2024-01-19	4000.00	4000.00	0.00	nochange
2024-01-20	2400.00	4000.00	-1600.00	decrease

Insights

"Analyzing daily sales trends helps identify demand patterns, detect anomalies, and support data-driven decision-making."

21. Calculate the running total of sales.

```sql
select
        o.order_date,
        SUM(oi.total_amount) as total_amount,
        SUM(SUM(oi.total_amount)) OVER (order by o.order_date) as runningtotal
from order_items oi
    join orders o
        on oi.order_id = o.order_id
    where o.order_id IN(
        select
                order_id
        from payments
        where LOWER(payment_status) = 'success'
    )
group by o.order_date
order by o.order_date;
```

order_date	total_amount	runningtotal
2024-01-01	70000.00	70000.00
2024-01-02	50000.00	120000.00
2024-01-04	3500.00	123500.00
2024-01-06	15000.00	138500.00
2024-01-07	6000.00	144500.00
2024-01-09	4000.00	148500.00
2024-01-10	4000.00	152500.00
2024-01-11	70000.00	222500.00
2024-01-12	50000.00	272500.00
2024-01-14	3500.00	276000.00
2024-01-16	15000.00	291000.00
2024-01-17	4000.00	295000.00
2024-01-19	4000.00	299000.00
2024-01-20	2400.00	301400.00

Insights

“Running total analysis helps track overall revenue progression and evaluate long-term business performance trends.”

22. Calculate month-over-month growth using LAG function.

```sql
SELECT
    YEAR(o.order_date) AS year,
    MONTH(o.order_date) AS month,
    SUM(oi.total_amount) AS monthly_sales,

    LAG(SUM(oi.total_amount)) OVER (
        ORDER BY YEAR(o.order_date), MONTH(o.order_date)
    ) AS prev_month_sales,

    SUM(oi.total_amount) - LAG(SUM(oi.total_amount)) OVER (
        ORDER BY YEAR(o.order_date), MONTH(o.order_date)
    ) AS growth

FROM orders o
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY
    YEAR(o.order_date),
    MONTH(o.order_date);
```

year	month	monthly_sales	prev_month_sales	growth
2024	1	754300.00	*NULL*	*NULL*
2024	2	63500.00	754300.00	-690800.00

Insights

"Month-over-month growth analysis helps identify business trends, measure growth rate, and detect periods of decline or improvement."

23. Rank customers based on total spending.

```sql
select
      c.customer_name,
      oi.order_id,
      SUM(oi.total_amount) as totalspend ,
      RANK() over (order by SUM(oi.total_amount) DESC) as ranking
from order_items oi

    join orders o
      on oi.order_id = o.order_id

    join customers c
      on o.customer_id = c.customer_id

    where o.order_id IN(
        select
              order_id
        from payments
        where LOWER(payment_status) = 'success'
    )
group by c.customer_name,oi.order_id
order by ranking;
```

customer_name	order_id	totalspend	ranking
Customer_1	1001	70000.00	1
Customer_11	1011	70000.00	1
Customer_12	1012	50000.00	3
Customer_2	1002	50000.00	3
Customer_6	1006	15000.00	5
Customer_16	1016	15000.00	5
Customer_7	1007	6000.00	7
Customer_9	1009	4000.00	8
Customer_10	1010	4000.00	8
Customer_17	1017	4000.00	8
Customer_19	1019	4000.00	8
Customer_4	1004	3500.00	12
Customer_14	1014	3500.00	12
Customer_20	1020	2400.00	14

Insights

“Ranking customers by total spending enables targeted marketing, prioritization of high-value customers, and improved customer retention strategies.”

24.Find the top product in each category.

```sql
select
        category,
        product_name
FROM
    (
        SELECT
            p.category,
            p.product_name,
            RANK() OVER(partition by p.category  order by SUM(oi.quantity) DESC) as ranking
        from products p
            join order_items oi
                on p.product_id = oi.product_id
            where oi.order_id IN(
                select
                    order_id
                from payments
                WHERE LOWER(payment_status) = 'success'
            )
            group by
                p.category,
                p.product_name
    )t
where ranking = 1;
```

category	product_name
Electronics	Headphones
Fashion	T-Shirt
Home	Dining Table
Sports	Cricket Bat

Insights

"Identifying top products in each category helps focus on high-demand items, optimize inventory, and improve category-wise sales strategy."

25.Identify the top 20% of customers contributing to 80% of total revenue (Pareto Analysis).

```sql
select * FROM(
    select
        c.customer_id,
        c.customer_name,
        SUM(oi.total_amount) as total_sales,
        ROUND(
          SUM(SUM(oi.total_amount)) over (order by SUM(oi.total_amount) DESC)
          * 100.00 /
          SUM(SUM(oi.total_amount)) over () ,
          2) as cumulative
    from order_items oi
    join orders o on oi.order_id = o.order_id
    join customers c on o.customer_id = c.customer_id
    where o.order_id IN(
        select
            order_id
            from payments
            where LOWER(payment_status) = 'success'
    )
    group by c.customer_id,c.customer_name
)t
where cumulative <= 80;
```

customer_id	customer_name	total_sales	cumulative
1	Customer_1	70000.00	46.450000
11	Customer_11	70000.00	46.450000
12	Customer_12	50000.00	79.630000
2	Customer_2	50000.00	79.630000

Insights

“Pareto analysis helps identify high-value customers who drive most of the revenue, enabling focused retention and targeted marketing strategies.”

Buisness insights

- A small group of high-value customers contributes significantly to total revenue, highlighting the importance of customer retention.
- Top-performing products and categories drive the majority of sales, indicating the need to focus on high-demand items.
- Sales trends (daily and monthly) reveal patterns and seasonal fluctuations, helping in better forecasting and planning.
- City-wise analysis shows that certain regions generate higher revenue, enabling targeted market expansion strategies.
- Customer behavior analysis indicates strong repeat purchasing, reflecting customer loyalty and engagement.
- Payment success and failure trends highlight potential revenue leakage and the need for improving payment systems.
- Unsold or low-performing products indicate opportunities for inventory optimization and better product positioning.
- Average order value indicates customer purchasing power and helps design pricing and bundling strategies.
- Order-to-customer ratio shows the level of customer engagement and repeat purchase behavior.
- Category-wise revenue distribution highlights which segments should receive more marketing and inventory focus.

Thank You